Dia 2 — Peano e indução no Lean

Folha de apoio e exercícios para casa

Hoje em uma frase

Hoje em uma frase

Indução no Lean é a indução do papel: um caso base e um passo — e o Lean confere os dois.

Hoje construímos os números naturais do zero, como Peano: um número é o *zero* ou o *sucessor* de outro número. A adição foi definida por recursão, e daí saíram fatos "por definição" (que o `rfl` fecha) e fatos que *não* são por definição, como $0 + n = n$, que pedem indução. Depois provamos, em cadeia, $0 + n = n$, $\text{succ}(n) + m = \text{succ}(n + m)$, a comutatividade e a associatividade — cada prova usando as anteriores, como num livro de álgebra.

Peano, em Lean

Os axiomas de Peano cabem em três linhas. Em Lean, isso é um *tipo indutivo*: a lista de todas as maneiras de construir um natural.

```
inductive Nat : Type where
  | zero : Nat              -- axioma 1: 0 é natural
  | succ (n : Nat) : Nat    -- axioma 2: todo n tem um sucessor
```

Não digite isso no Lean Web: `Nat` já existe, e o Lean reclama de nome repetido. Para ver a definição que já está lá, digite `#print Nat`. Os axiomas 3, 4 e 5 (zero não é sucessor; sucessor é injetivo; indução) *não precisam ser escritos*: o Lean os deduz da própria forma da definição. Eles aparecem como os teoremas `Nat.succ_ne_zero`, `Nat.succ.inj` e como a tática `induction`.

A adição e o que é "por definição"

A adição é definida por recursão **no segundo argumento**, com duas equações:

$$n + 0 = n, \qquad n + \text{succ}(m) = \text{succ}(n + m).$$

No Lean elas já estão provadas com os nomes `Nat.add_zero` e `Nat.add_succ`. "Por definição" quer dizer: o Lean consegue *calcular* um lado até chegar no outro, usando só essas equações. É isso que o `rfl` faz.

Enunciado	`rfl` fecha?	Por quê
`n + 0 = n`	sim	é a 1.ª equação da definição
`n + Nat.succ m = Nat.succ (n + m)`	sim	é a 2.ª equação
`(2 : ℕ) + 2 = 4`	sim	números concretos: o Lean calcula tudo
`Nat.succ n = n + 1`	sim	$n+1 = n+\text{succ}(0) = \text{succ}(n+0) = \text{succ}(n)$
`0 + n = n`	**não**	a recursão é no 2.º argumento, e n é desconhecido: nada a calcular

A última linha é a assimetria da definição no papel: $n + 0$ "desce" pela recursão; $0 + n$, com n genérico, não desce. Precisamos de indução.

A tela da indução, parte por parte

Esta é a prova de $0 + n = n$ que fizemos juntos (o nome tem prefixo `meu_` porque a Mathlib já tem um `zero_add`, e o Lean não aceita dois teoremas com o mesmo nome):

```
theorem meu_zero_add (n : ℕ) : 0 + n = n := by
  induction n with
  | zero => rfl                -- caso base
  | succ n ih =>               -- passo indutivo
    rw [Nat.add_succ]
    rw [ih]
```

Com o cursor logo depois de `| succ n ih =>`, o Infoview mostra exatamente isto:

```
case succ
n : ℕ
ih : 0 + n = n
⊢ 0 + (n + 1) = n + 1
```

Linha	Nome	O que é
`case succ`	o caso	Estamos no passo indutivo (o outro caso, `zero`, é a base).
`n : ℕ`	objeto	"Seja n um natural."
`ih : 0 + n = n`	hipótese de indução	O que o passo nos dá: a afirmação vale para n. O nome `ih` fomos nós que escolhemos.
`⊢ 0 + (n + 1) = n + 1`	meta	O que falta: a afirmação para o sucessor de n.

Repare: o Lean escreve o sucessor como `n + 1`, não como `Nat.succ n`. São o mesmo número; é só a forma de exibir.

Analogia: a indução que você já faz no papel

- `induction n with` = "Provemos por indução em n."
- `| zero =>` = "*Caso base.*" A meta vira a afirmação para 0.
- `| succ n ih =>` = "*Passo.* Suponha que vale para n (hipótese `ih`); mostremos para $n+1$."
- Cada `rw` = uma igualdade da sua cadeia, com a justificativa entre parênteses.
- `No goals` = fim da demonstração.

A diferença: no papel, quem confere a indução é o leitor. Aqui, o Lean confere os dois casos e não deixa você esquecer nenhum.

Um passo, visto de perto

No passo indutivo, a prova no papel é $0 + (n+1) \overset{\text{def}}{=} (0+n) + 1 \overset{\text{HI}}{=} n + 1$. Em Lean, uma linha por igualdade:

```
⊢ 0 + (n + 1) = n + 1     -- antes
rw [Nat.add_succ]         -- usa  n + succ m = succ (n + m)
⊢ (0 + n).succ = n + 1    -- depois: o "+1" saiu para fora
rw [ih]                   -- usa  ih : 0 + n = n
                          -- a meta vira  succ n = n + 1, que é igual
                          -- por definição: o rw fecha sozinho → No goals
```

`(0 + n).succ` é a forma como o Lean escreve $\mathrm{succ}(0+n)$. O `rw [ih]` procura `0 + n` na meta e troca por `n`, exatamente como você substitui usando a hipótese de indução.

Lema × Tática

`Nat.add_succ` é um **lema**: um resultado já demonstrado, que você cita ("pela definição da soma"). `rw` é a **tática**: o ato de usar uma igualdade para substituir. Na linha `rw [Nat.add_succ]`, o lema é `Nat.add_succ` e o ato é `rw`. Outras táticas de hoje:

- `rewrite [h]`: só substitui. `rw [h]` substitui e depois tenta `rfl` (por isso o `rw [ih]` acima já fechou a prova).
- `rw [← h]`: usa a igualdade `h` da direita para a esquerda.
- `rw [a, b, c]`: várias substituições em sequência, numa linha só.

Quando o Lean recusa

Antes da indução, tentamos `rfl` direto em $0 + n = n$. O Lean responde (em vermelho):

```
Tactic `rfl` failed: The left-hand side
  0 + n
is not definitionally equal to the right-hand side
  n
```

Leia assim: "não consigo calcular `0 + n` até chegar em `n`" — é exatamente a última linha da tabela da página anterior. Uma mensagem de erro não é punição: é o Lean dizendo *qual* passo não se justifica. A meta continua lá, esperando outro passo.

Uma prova usa a anterior

Com `meu_zero_add` pronto, provamos o "primo" dele, agora por indução em m:

```
theorem meu_succ_add (n m : ℕ) : Nat.succ n + m = Nat.succ (n + m) := by
  induction m with
  | zero => rfl
  | succ m ih =>
    rw [Nat.add_succ]    -- ⊢ (n.succ + m).succ = (n + (m + 1)).succ
    rw [ih]              -- ⊢ (n + m).succ.succ = (n + (m + 1)).succ
    rw [Nat.add_succ]    -- os dois lados ficam iguais → No goals
```

E, com os dois, a comutatividade (indução em n):

```
theorem meu_add_comm (n m : ℕ) : n + m = m + n := by
  induction n with
  | zero =>                              -- ⊢ 0 + m = m + 0
    rw [Nat.add_zero, meu_zero_add]
  | succ n ih =>                         -- ⊢ n + 1 + m = m + (n + 1)
    rw [meu_succ_add, ih, Nat.add_succ]
```

No papel: $\text{succ}(n) + m = \text{succ}(n + m)$ (`meu_succ_add`) $= \text{succ}(m + n)$ (HI) $= m + \text{succ}(n)$ (`Nat.add_succ`). A hierarquia é a de um livro: `meu_add_comm` depende de `meu_zero_add` e `meu_succ_add`, que dependem das duas equações da definição.

Estratégia (do slide)

1. Leia o enunciado e reconheça a forma lógica.
2. É verdade por definição? Tente `rfl`.
3. Tem uma variável e a definição é recursiva? Pense em `induction` — em qual variável? Na que aparece como *segundo* argumento da soma.
4. Veja quais lemas já estão provados e use `rw` com eles.
5. Identidade algébrica, fora dos exercícios de Peano? `ring` ou `omega` fazem sozinhos (são "atalhos": ótimos no dia a dia, mas nos exercícios de Peano a graça é não usá-los).

Para onde isso vai: pesquisa e ensino

Pesquisa. Hoje construímos tudo do zero. Na pesquisa, ninguém faz isso: a Mathlib já tem centenas de milhares de definições e teoremas. No fim da aula vimos que "se f e g são contínuas em a, então $f + g$ é contínua em a" tem uma prova ε–δ de várias linhas em Lean — e que a Mathlib já tem esse resultado pronto, com o nome `ContinuousAt.add`. Táticas como `exact?` procuram o lema certo para você. Não precisa entender agora: o ponto é que você não recomeça do zero.
Ensino. Repare como cada prova de hoje ficou lado a lado com a prova no papel, uma linha para cada igualdade. Para um aluno de primeiro ano, o Lean é um corretor que não se cansa: ele recusa o passo que não se justifica e mostra o que falta. Há disciplinas introdutórias de demonstração que usam o próprio Natural Number Game como exercício.

Glossário acumulado (Dias 1–2)

Palavra	Em uma linha — e o análogo no papel
Proposição (`Prop`)	Um enunciado, verdadeiro ou falso. *O enunciado do teorema.*
Tipo / `a : A`	Toda coisa no Lean tem um tipo; `n : ℕ` lê-se "n é um natural". *"Seja $n \in \mathbb{N}$".*
Prova = termo	Provar P é construir um objeto do tipo P (Curry–Howard). *A demonstração como objeto.*
Kernel	A parte pequena do Lean que confere cada prova. *O leitor rigoroso que confere cada linha.*
`theorem` / `example`	Teorema com nome / sem nome. *"Teorema 3"* / *"Exemplo".*
`def`	Define um objeto ou uma função. *"Definição".*
`#check` / `#eval`	Mostra o tipo (o enunciado) / calcula o valor. *Consultar o livro / fazer a conta.*
Meta (`⊢`)	O que falta provar. *A tese, no quadro.*
Hipótese	Fato disponível, com nome (`h`, `ih`). *O que o enunciado (ou a indução) nos dá.*
Tática	Um comando que transforma a meta. *Um passo justificado da demonstração.*
Lema	Resultado já provado, citável (`Nat.add_succ`). *O resultado do livro que você cita.*
`No goals`	Nada mais a provar. ■.
`sorry`	"Prova pendente": aviso amarelo; o teorema *não* está provado. *"Exercício para o leitor".*
`inductive`	Define um tipo listando as formas de construir seus elementos. *Os axiomas de Peano.*
`Nat.zero`, `Nat.succ`	O zero e o sucessor. 0 *e* $S(n)$.
Recursão	Definir $f(\text{succ}\, m)$ a partir de $f(m)$. *Definição por recorrência.*
Por definição	O Lean calcula um lado até o outro. *"Imediato da definição".*
`rfl`	Fecha metas verdadeiras por definição. *"É imediato".*
`rewrite [h]`	Substitui usando a igualdade `h`. *"Por (h), temos..."*
`rw [h]`	`rewrite` + tentativa de `rfl`. *Substituir e ver se acabou.*
`rw [← h]`	Usa `h` da direita para a esquerda. *Ler a igualdade ao contrário.*
`induction n with`	Divide em caso base e passo. *"Por indução em n".*
Caso base (`zero`)	A meta para $n = 0$. *"Para $n = 0$..."*
Passo (`succ n ih`)	Supondo `ih` para n, provar para $n + 1$. *"Suponha que vale para n..."*
`ih`	Hipótese de indução (o nome é escolha sua). *HI.*
`have h : P := ...`	Prova um fato auxiliar no meio da prova. *"Afirmação: ..."*
`simp`, `omega`, `ring`	Automação: simplifica / aritmética linear em $\mathbb{N}$ / álgebra. *"Contas de rotina".*
`exact h`	Fecha a meta com `h`, que é exatamente o que se pede. *"...que é a hipótese."*

Como digitar os símbolos: $\mathbb{N}$ = `\N` $\leftarrow$ = `\l` $\to$ = `\to` $\neq$ = `\ne` $\exists$ = `\exists` $\langle\ \rangle$ = `\<` e `\>` (digite e aperte espaço). No Lean Web, passar o mouse sobre um símbolo mostra como digitá-lo.

Exercícios para casa

Faça na ordem; o total deve levar de 1h a 1h30. Regra de ouro: **antes de apertar Enter, escreva a meta que você espera ver**, e só depois confira. Acertar a previsão é o que mostra que você entendeu. Nos exercícios no Lean Web, comece o arquivo com `import Mathlib` (a primeira carga demora um pouco). Não use `omega`, `ring` nem `simp` nos exercícios 2 a 5. As dicas estão no fim da folha: tente antes de olhar.

Exercício 1 — A comutatividade como texto de livro (papel)

Reescreva a prova de `meu_add_comm` (seção "Uma prova usa a anterior") como você a escreveria num livro: "Por indução em n. *Caso base:* ... *Passo:* ...". Uma igualdade por linha, e em cada linha, entre parênteses, a justificativa (a equação da definição, a HI ou o resultado anterior que foi usado). Depois responda: por que a indução é em n, e não em m? (Dica: não é obrigatório; mas pense em qual lema anterior ficou útil.) **Traga:** o texto.

Exercício 2 — Prever antes de apertar (NNG, Addition World: zero_add e succ_add)

No Natural Number Game, entre no *Addition World* e faça os níveis `zero_add` e `succ_add` (os dois primeiros). Em cada um:

(a) antes de `induction n with d hd`, escreva as *duas* metas que você espera ver (caso base e passo) e qual hipótese nova vai aparecer;

(b) antes de cada `rw`, escreva a meta seguinte; confira;

(c) no `succ_add`: em qual variável fazer a indução? Justifique pela definição da soma.

(Na sua tela, `succ d` pode aparecer escrito de outro jeito; a ideia é a mesma.) **Traga:** as previsões, marcando as que erraram.

Exercício 3 — Completar a associatividade (Lean Web)

Cole no Lean Web e troque cada `sorry` por táticas, até aparecer `No goals`.

```
import Mathlib

theorem meu_add_assoc (n m k : ℕ) : n + m + k = n + (m + k) := by
  induction k with
  | zero => sorry         -- use a definição da soma
  | succ k ih => sorry    -- use Nat.add_succ (quantas vezes?) e ih
```

Antes, escreva a prova no papel: "Por indução em k...". (Lembre que `n + m + k` quer dizer `(n + m) + k`.) Enquanto houver `sorry`, o Lean mostra aviso amarelo: o teorema *ainda não* está provado. **Traga:** a prova no papel e o código final.

Exercício 4 — Erro de propósito (Lean Web)

Digite esta versão de `0 + n = n`, com as duas linhas do passo *trocadas de ordem*:

```
example (n : ℕ) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    rw [ih]
    rw [Nat.add_succ]
```

(a) Copie a mensagem de erro e a meta que aparece junto com ela.

(b) Explique em uma frase, olhando para a meta, por que o `rw [ih]` falhou.

(c) Conserte a prova e diga qual é o "passo do papel" que precisa vir primeiro.

Traga: (a), (b) e (c).

Exercício 5 — A comutatividade sozinho (NNG, Addition World: add_comm)

No Natural Number Game, faça o nível `add_comm` ($a + b = b + a$) **sem consultar esta folha**. Você pode usar os lemas que provou nos níveis anteriores (`zero_add`, `succ_add`). Se não sair, tudo bem: anote até onde chegou e qual era a meta na tela. **Traga:** a solução, ou a meta em que travou.

Exercício 6 — A IA propõe, o Lean julga (chat de IA + Lean Web)

Peça a um chat de IA (Claude, Gemini, ChatGPT...): *"Escreva uma prova em Lean 4, com Mathlib, de que $2 \cdot n = n + n$ para todo natural n."*

(a) Cole a resposta no Lean Web (com `import Mathlib` no topo). O Lean aceitou? Se recusou, copie a primeira mensagem de erro.

(b) Peça ao chat para explicar, linha por linha, o que cada tática faz. A explicação bate com o que o Infoview mostra quando você põe o cursor em cada linha?

(c) Em uma frase: quem você deve considerar como a palavra final — o chat ou o Lean? Por quê?

Traga: a prova proposta, o veredito do Lean e sua frase.

Exercício 7 — Desafio opcional: ímpar + ímpar = par (Lean Web)

Complete a prova abaixo. Aqui *pode* usar `omega` para a conta final.

```
import Mathlib

def Par (n : ℕ) : Prop := ∃ x, n = x + x
def Impar (n : ℕ) : Prop := ∃ x, n = x + x + 1

theorem impar_add_impar {a b : ℕ} (ha : Impar a) (hb : Impar b) :
    Par (a + b) := by
  sorry
```

Alternativa, para quem prefere o jogo: siga no NNG pelo *Multiplication World* até o nível `mul_comm`.

Se travar

1. **Releia a meta** (a linha com ⊢). O que está de um lado que não está do outro?
2. **Olhe as hipóteses e os lemas disponíveis.** Algum tem o lado esquerdo parecido com um pedaço da meta? (No NNG, a lista de lemas e táticas liberadas fica ao lado.)
3. **Escreva a prova no papel primeiro.** Cada igualdade da sua cadeia vira um `rw`.
4. **No NNG, use o botão de dicas do próprio jogo.**
5. **Pergunte a um chat de IA pedindo explicação, não a resposta.** Modelo: *"Estou no Natural Number Game, Addition World, nível succ_add. Minha meta é* [*cole aqui*] *e minhas hipóteses são* [*cole aqui*]. *Não me dê a solução: explique o que a tática rw faria aqui e me faça uma pergunta que me ajude a pensar."* Para o Lean Web, troque o início por *"Estou no Lean 4 com Mathlib, recebi esta mensagem de erro:* [*cole aqui*]. *Explique o que ela quer dizer, sem corrigir a prova por mim."* (A IA pode errar em detalhes — nomes de lemas, sintaxe antiga do Lean 3; quem decide é o Lean.)
6. **Anote a dúvida**, com a meta que estava na tela, e traga. Travar é informação, não fracasso.

Para continuar

- **Natural Number Game** (`adam.math.hhu.de`): depois do Addition World, siga para Multiplication, Power, Implication, Advanced Addition e ≤.
- **Mathematics in Lean** (MIL) — o melhor próximo passo para matemáticos: `https://leanprover-community.github.io/mathematics_in_lean`.
- **Theorem Proving in Lean 4** (TPIL), para entender a lógica por trás: `https://leanprover.github.io/theorem_proving_in_lean4`.
- **Documentação da Mathlib**: `https://leanprover-community.github.io/mathlib4_docs`.
- **Comunidade**: Zulip `leanprover.zulipchat.com`, canal *#new members* (muito receptivo a iniciantes).
- **Instalar no seu computador** (quando quiser ir além do Lean Web): VS Code + extensão "Lean 4" (lean-lang.org → Install).

Dicas graduadas

Leia só a Dica 1; tente de novo; só então passe para a próxima.

Exercício 1

Dica 1

Caso base: é preciso mostrar $0 + m = m + 0$. Cada lado se resolve com um fato diferente: um é a definição da soma, o outro é um resultado que provamos antes.

Dica 2

Passo: suponha $n + m = m + n$ (HI) e mostre $\text{succ}(n) + m = m + \text{succ}(n)$. Comece pelo lado esquerdo e use `meu_succ_add`.

Quase a resposta

Base: $0 + m = m$ (`meu_zero_add`) $= m + 0$ (definição). Passo: $\text{succ}(n) + m = \text{succ}(n + m)$ (`meu_succ_add`) $= \text{succ}(m + n)$ (HI) $= m + \text{succ}(n)$ (definição). Sobre n ou m: fazendo em n, o passo pede exatamente `meu_succ_add`; fazendo em m, o passo pediria succ no segundo argumento de um lado e no primeiro do outro — também dá, com os mesmos lemas em outra ordem.

Exercício 2

Dica 1

No `zero_add`, depois da indução, a base é `0 + 0 = 0` e o passo é `0 + succ d = succ d`, com a hipótese `hd : 0 + d = d`.

Dica 2

No `succ_add` (succ $a + b = \text{succ}(a + b)$) a indução é em `b`: a soma é definida por recursão no *segundo* argumento, e `b` é o segundo argumento dos dois lados.

Quase a resposta

`zero_add`: `induction n with d hd`; base: `rw [add_zero], rfl`; passo: `rw [add_succ]` (meta `succ (0 + d) = succ d`), `rw [hd]` (meta `succ d = succ d`), `rfl`. `succ_add`: `induction b with d hd`; base: `rw [add_zero], rw [add_zero], rfl`; passo: `rw [add_succ], rw [add_succ], rw [hd], rfl`.

Exercício 3

Dica 1

Caso base: a meta é `n + m + 0 = n + (m + 0)`. Os dois lados são iguais *por definição*.

Dica 2

Passo: a meta é `n + m + (k + 1) = n + (m + (k + 1))`. Há três lugares com "algo + sucessor". Use `rw [Nat.add_succ]` até sumirem; aí use `ih`.

Quase a resposta

Base: `rfl`. Passo: `rw [Nat.add_succ, Nat.add_succ, Nat.add_succ, ih]`. As metas intermediárias são `(n + m + k).succ = n + (m + (k + 1))`, depois `(n + m + k).succ = n + (m + k).succ`, depois `(n + m + k).succ = (n + (m + k)).succ`, e o `rw [ih]` fecha.

Exercício 4

Dica 1

A mensagem diz que o Lean não achou um padrão (`0 + n`) dentro da meta. Procure `0 + n`, literalmente, na meta `0 + (n + 1) = n + 1`.

Dica 2

`0 + (n + 1)` não contém `0 + n` como pedaço: o $+1$ está "dentro". Qual lema põe o $+1$ para fora?

Quase a resposta

A mensagem é (algo como) `Did not find an occurrence of the pattern 0 + n in the target expression 0 + (n + 1) = n + 1`. O `rw [ih]` só troca `0 + n` se esse pedaço aparecer na meta. Primeiro vem a definição da soma, `rw [Nat.add_succ]`, que transforma a meta em `(0 + n).succ = n + 1`; só então a HI tem onde se encaixar. No papel: primeiro "$0 + (n + 1) = (0 + n) + 1$ (def.)", depois "$= n + 1$ (HI)".

Exercício 5

Dica 1

Faça indução em `b`.

Dica 2

Base: `a + 0 = 0 + a`. Um lado se resolve com `add_zero`, o outro com `zero_add`. Passo: `add_succ` de um lado, `succ_add` do outro, e `hd` no meio.

Quase a resposta

`induction b with d hd`; base: `rw [add_zero]`, `rw [zero_add]`, `rfl`; passo: `rw [add_succ]`, `rw [succ_add]`, `rw [hd]`, `rfl`.

Exercício 6

Dica 1

É comum a IA usar nomes de lemas que não existem, ou sintaxe do Lean 3 (`begin ... end`, `nat.succ`). Se o Lean recusar, a mensagem normalmente aponta a linha exata.

Dica 2

Se a IA usou uma tática automática (`omega`, `ring`, `simp`), pergunte a ela o que essa tática faz e por que ela resolve o problema.

Quase a resposta

Uma prova que o Lean aceita é `example (n : ℕ) : 2 * n = n + n := by omega`. Sua frase deve dizer algo como: "a palavra final é do Lean, porque ele confere cada passo com regras fixas; a IA propõe, e pode errar".

Exercício 7

Dica 1

`ha : Impar a` diz "existe x com $a = x+x+1$". Para usar um "existe", dê nome à testemunha: `obtain ⟨x, hx⟩ := ha` ("*seja x tal que...*").

Dica 2

Para provar `Par (a + b)` você precisa dar a testemunha e a prova da igualdade: `⟨testemunha, prova⟩`. No papel: $(2x + 1) + (2y + 1) = 2(x + y + 1)$.

Quase a resposta

Troque o `sorry` por:

```
obtain ⟨x, rfl⟩ := ha      -- a = x + x + 1
obtain ⟨y, rfl⟩ := hb      -- b = y + y + 1
exact ⟨x + y + 1, by omega⟩
```

(Com `rfl` no lugar de `hx`, o Lean já substitui a por `x + x + 1` na meta.)